

Exercise 5: Sagas and Stateful Resilience Patterns

Deadline: 16.04.2026, 23:59

Group 1, Team Members:

Michael Schütz, Gianluca Ielpo, Eva Amromin

Project Overview

This project coordinates ordering, inventory, and factory execution, the project overview is shown below.

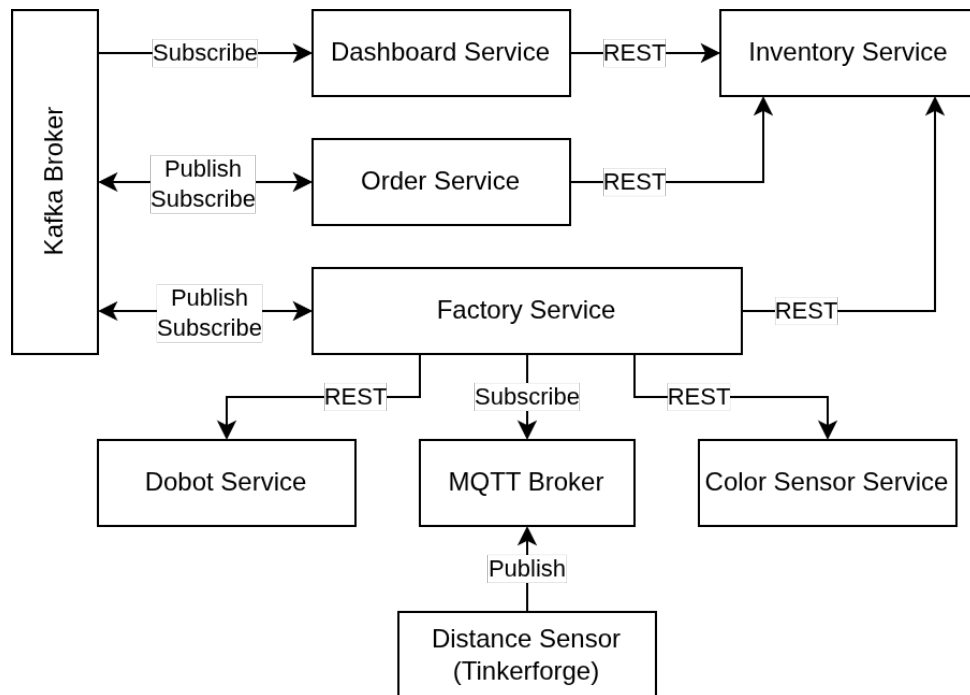


Figure 1: Block Diagram

Successful Order Fulfillment

The diagram shows the normal path from order submission to completion.

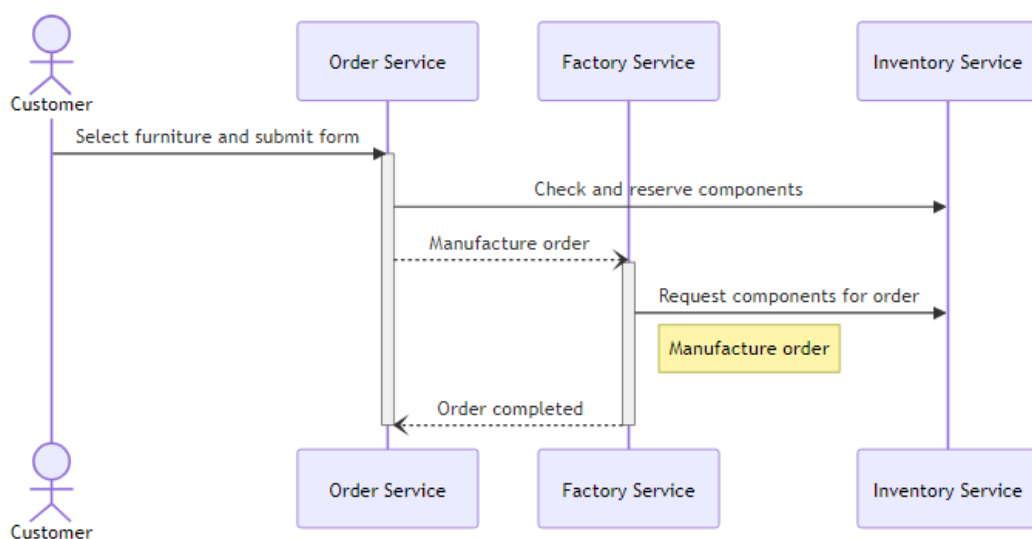


Figure 2: Sequence Diagram

The flow begins when a customer selects a single furniture item and submits the order form through Camunda Forms in the Order Service. The Order Service, acting as the orchestrator, immediately calls the Inventory Service via REST to reserve the required components. If the reservation succeeds, the Order Service publishes a manufacture command to the `order.manufacture.v1` Kafka topic and then waits at an event-based gateway, listening for either a completion event, an error event, or a 10-minute timeout.

The Factory Service consumes the command from Kafka and starts its own BPMN process. It first calls the Inventory Service via REST to retrieve the physical locations of the reserved components. Once it has the component data, it emits a status event to `info.v1` to signal that manufacturing has started, then drives the Dobot robot through the assembly steps, picking and placing each block. When assembly is complete, the Factory Service publishes an event to `order.complete.v1`.

The Order Service receives the `order.complete.v1` event, which unblocks the waiting gateway. It then publishes a success status event to `info.v1` and the process ends. The Dashboard, which continuously consumes from `info.v1`, reflects the updated order status to the customer in real time.

Dobot Control Service

github.com/ielpo/edpo-project-group1/services/dobot-control

The robot is controlled via serial using the `pydobotplus` Python library. The existing control service was very limited and did not fulfill the requirements for this project, therefore it was refactored and improved. The new implementation supports a simulation mode, in which the communication with the robot is faked and always succeeds.

Additionally, the service allows to execute individual commands including relative coordinate movements, which are required in order to fetch items in the inventory grid and to place the blocks in the assembly area with precision.

Color Sensor Service

github.com/ielpo/edpo-project-group1/services/color-sensor

github.com/ielpo/edpo-project-group1/services/color-sensor-fake

The existing color sensor of the Dobot system is very limited and only returns a boolean for each color, rendering the detection of all four colors infeasible. To mitigate these issues, a new color sensor was implemented on a raspberry pi pico using a TCS34725. The WiFi interface on the Pico allows the color sensor to offer a REST interface.

To ease debugging and automated testing, a fake color sensor service is implemented in Rust.

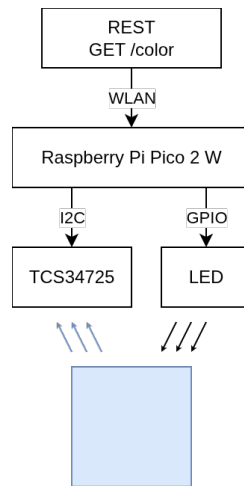


Figure 3: Color Sensor

Distance Sensor

The distance sensor is part of the Tinkerforge setup, and publishes messages over MQTT after each measurement. The distance is used to detect if a block was picked up, and avoid issues where the block is lost during transfer or the inventory service had a wrong internal state compared to the physical inventory.

Stateful resilience patterns

The following resilience patterns are used throughout the Kafka Project.

Stateful retry

If a downstream service is briefly unavailable, we don't want to immediately push that error back to the user. Instead, we use Operator's built-in job retry mechanism to handle transient failures automatically.

We applied this in three places in the Order process:

- *Reserve inventory* — when an order comes in
- *Reset inventory* — during compensation after a timeout
- *Reset inventory* — during compensation after a manufacturing error

All three run async with *R3/PT10S*, so the engine retries up to three times, ten seconds apart. The retry state is persisted by the workflow engine, meaning it survives restarts. The caller never has to deal with a "try again later" for what's just a temporary issue.

When retries are genuinely exhausted, our workers check if it's the last attempt and raise a proper BPMN error (e.g. *INVENTORY_SERVICE_UNAVAILABLE*), so the process can react through its error or compensation paths.

Human Intervention Pattern

The human intervention pattern is used for recovery steps that are very hard to fully automate. In our order process, if manufacturing times out or fails, we don't just continue blindly with automation as the state of the physical inventory is unknown. Instead, the process waits in a user task (*Restock inventory*, assignee *demo* (used for convenience)) so a

person can return the physical inventory to its original state and confirm the action with a tiny Camunda form.

Only after this manual step is completed we continue with the technical restore flow. This keeps process control explicit: automation handles technical issues, while operational recovery is delegated to a human when the situation is too complex to handle for the system. As a simplification, the process then ends with an error rather than restarting the order.

Epic Saga Pattern

The overall flow follows the traditional *Epic Saga* idea with one clear orchestrator. In our case, the Order service coordinates the transaction-like business request and monitors whether all required steps complete.

If a later step fails, we do not leave already performed actions as-is. Instead, we trigger compensating actions (most importantly inventory restore) to reverse earlier writes inside the distributed transaction scope. However, true transaction isolation across services is not guaranteed as intermediate state changes may be visible to other parts of the system before a rollback occurs, and compensating actions themselves could potentially fail.

Lessons Learned

- Human intervention is essential for systems like ours where real-world state cannot always be reconstructed from software state alone.
- Being able to develop software independently from the hardware is of advantage, it allows for quicker iteration.
- Error handling by Operaton requires to throw specific exeptions, else the retry mechanism is triggered. In our case this led to the factory trying to fetch blocks for the same order multiple times.

Reflections

What worked well:

- Separating orchestration logic (Order) from execution logic (Factory) kept responsibilities understandable.
- Combining automation with manual checkpoints made recovery more realistic for physical processes.
- The services were implemented individually and tested in isolation, which made development more manageable and allowed us to validate each part before integration.
- The integration was straightforward due to the clear contract of Kafka topics and REST APIs.
- Hexagonal Architecture worked very well for keeping boundaries and dependencies explicit.

What was challenging:

- The initial Camunda setup was a bit tricky to get right due to all the different required dependencies and configurations.

- Most of the project has fairly little business logic while Hexagonal Architecture required many interfaces and adapters, which felt like substantial boilerplate.
- Access to the lab is limited, limiting the time spent developing with hardware.

What we would improve next:

- Replace fixed retry policies with context-aware retry/backoff per failure type.
- Add more details to the Kafka events (e.g. failure reasons, retry counts) to support better observability and debugging. This would have also made the dashboard simpler to implement.
- Add testing (unit and integration).
- Implement a proper simulation mode for the factory that can be used for automated testing.

ADRs

The following ADRs are related to this exercise:

- ADR 0009: Granularity of Factory Service
- ADR 0014: Hexagonal Architecture for Order and Factory Services.
- ADR 0015: Embedded H2 Database for Process Engine Persistence.
- ADR 0016: Correlation ID Strategy.

Contributions

Person	Tasks
Michael	Order service, Factory service support, ADRs, Documentation, Presentation
Eva	Inventory service, Dashboard, Order service support, ADRs, Documentation, Presentation
Gianluca	Factory and factory service, ADRs, Documentation, Presentation